# Excel Shrink: An Open-Source Tool for Reducing File Bloat in Excel Spreadsheets

ANONYMOUS AUTHOR(S)
SUBMISSION ID: 1503

Excel files are widely used for data exchange across various domains. However, as spreadsheets grow in complexity, they often suffer from file bloat, leading to significant challenges in file size and performance management. We investigated Excel files from the German Federal Statistical Office (Destatis) and observed extremely long load times in Microsoft Excel and crashes due to out-of-memory exceptions in software packages used for reading Excel files. Our analysis revealed that these files contain format definitions invisible to the user, which occupy orders of magnitude more storage space than the actual data.

While some existing tools attempt to address this issue, they often fail to fully resolve the problem or introduce new issues, such as altering the visual appearance of spreadsheets. In this paper, we present a comprehensive analysis of the root causes of Excel file bloat, documenting issues that have been partially addressed by existing tools but remained undocumented, and uncovering additional issues not resolved by these tools. We also analyze why these tools impact the visual integrity of Excel files.

We developed an open-source tool, Excel Shrink, and its graphical user interface counterpart, Excel Shrink UI, which effectively address both the known and previously unresolved root causes of Excel file bloat without compromising the visual appearance of the spreadsheets. Excel Shrink UI enables users to optimize their Excel files without the need for programming knowledge or command-line tools. To evaluate our tools, we downloaded all Excel files from Destatis and optimized them using Excel Shrink. Our results show that our tools can reduce file sizes by up to 99.99% without affecting the visual integrity of the files.

CCS Concepts: • **Applied computing** → **Spreadsheets**; • **Information systems** → **Data cleaning**; • **Software and its engineering** → **Extensible Markup Language (XML)**; • **Human-centered computing** → *Graphical user interfaces*; • **Theory of computation** → Data compression.

Additional Key Words and Phrases: Excel File Optimization, XML Processing, Data Compression, Web Scraping, File Size Reduction, XLSX Format, Data Cleaning, Data Preprocessing, Open Data, Spreadsheet Efficiency, Data Management, Destatis

**ACM Reference Format:**
Anonymous Author(s). 2025. Excel Shrink: An Open-Source Tool for Reducing File Bloat in Excel Spreadsheets. In *Proceedings of 2025 International Conference on Management of Data (SIGMOD '25).* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

## 1 INTRODUCTION

Excel spreadsheets are integral to a wide range of domains—including finance, statistics, business, and research—due to their flexibility in managing tabular data, executing complex calculations, automating workflows via macros, and visualizing information using charts and graphs [15, 34, 35]. However, as these spreadsheets grow in size and complexity, they often suffer from performance bottlenecks, file bloat, and increased processing delays [13, 14, 39, 40].


*SIGMOD '25, June 22–27, 2025, Berlin, Germany*

ACM ISBN 978-1-4503-XXXX-X/25/06…$15.00
https://doi.org/10.1145/XXXXXXX.XXXXXXX

Unlike simpler formats such as CSV, Excel files leverage the Office Open XML architecture introduced in Excel 2007 [22]. Under this format, data, formatting, shared strings, and other metadata are stored as a collection of XML files, all compressed into a single ZIP archive. This design supports advanced features, such as separating data from formatting—e.g., centralizing repeated strings in a shared string table [17], or storing fonts, cell styles, and colors in dedicated XML files. While this architecture enables functionality beyond what simpler formats provide, it also creates opportunities for hidden inefficiencies to emerge.

### 1.1 Excel's Internal Storage Mechanisms and Resulting Inefficiencies

The ZIP-based compression of XML data efficiently reduces file size by exploiting repetitive tags and structures [20]. Yet, this approach can mask significant redundancies. Over time, spreadsheets can accumulate unnecessary formatting or metadata: for instance, entire rows or columns may be formatted even though only a subset of cells contain data [1, 18]. Similarly, duplicating a sheet transfers all formatting, styles, and hidden elements, even when most of them are not needed [23]. Such practices lead to inflated files, slower load times, and degraded overall performance [23, 36], while also increasing token usage in Large Language Model (LLM)-based workflows [41]. These issues become especially pronounced in large-scale enterprise environments or when handling extensive datasets, such as those provided by the German Federal Statistical Office (Destatis).

### 1.2 Limitations of Existing Solutions

Several proprietary tools and features, such as NXPowerLite Desktop, the Inquire Add-in, and the Excel Performance tab, attempt to mitigate file bloat. However, they suffer from notable shortcomings:

- **Closed Source and Lack of Transparency:** Existing tools do not disclose their underlying methods, making it difficult to understand the root causes of bloat or to improve upon their techniques.
- **Incomplete Coverage:** They often fail to address all sources of unnecessary content. For example, redundant column definitions may remain uncorrected, leaving substantial portions of file bloat unresolved.
- **Visual Alterations:** Some solutions can significantly alter the spreadsheet's original appearance, which is unacceptable for users who rely on specific formatting or corporate branding.
- **Limited Automation:** Due to the lack of batch processing or command-line interfaces, integrating these tools into automated workflows or pipelines (e.g., for cloud uploads or large-scale data processing) is challenging or impossible.

### 1.3 Our Approach and Contributions

Motivated by these gaps, we conducted a systematic investigation into the root causes of Excel file bloat, focusing on previously undocumented inefficiencies and the limitations of existing solutions. We then developed *Excel Shrink*, an open-source tool designed to effectively reduce file sizes while preserving the spreadsheet's visual appearance [10]. Unlike proprietary offerings, *Excel Shrink* provides a transparent, platform-independent solution that does not require Microsoft Excel itself. It can also be integrated into automated pipelines, enabling seamless batch processing and large-scale deployment. For those preferring a more user-friendly interface, we offer *Excel Shrink UI* [11], which eliminates the need for Python installation or command-line interactions.

Our contributions are as follows:

- **Comprehensive Analysis:** We reveal the root causes of file bloat in Destatis Excel files, identifying previously undocumented issues such as unnecessary column definitions.
- **Critical Evaluation of Existing Tools:** We show why current proprietary solutions fail to fully resolve file bloat and how they may inadvertently alter the spreadsheet's appearance.
- **Novel Open-Source Solution:** We introduce *Excel Shrink*, which effectively addresses all identified causes of bloat, preserves visual formatting, supports automation, and is not tied to a specific operating system or to Microsoft Excel.
- **Empirical Validation:** We evaluate *Excel Shrink* on a comprehensive set of Destatis Excel files, demonstrating that it can reduce file sizes by up to 99.99% without compromising usability or appearance.

By thoroughly examining hidden redundancies and providing a transparent, open-source solution, our work fills a significant gap in spreadsheet optimization research and practice.

## 2 RELATED WORK

Several tools and techniques have been developed to reduce *Excel* file bloat, including proprietary solutions such as *NXPowerLite Desktop*, the *Excel Inquire* Add-in, and the built-in *Check Performance* feature in recent versions of *Excel*. While these tools can alleviate certain issues, they also exhibit notable limitations.

### 2.1 NXPowerLite Desktop

*NXPowerLite Desktop* is a commercial application primarily focused on compressing embedded images in *Excel* files [25]. Although it can remove some empty cells, it disregards other sources of file bloat such as cells containing only whitespace and redundant column definitions.

The tool offers limited batch functionality, allowing users to select and process multiple files simultaneously, but only up to a maximum of 20 files at once. Being closed-source, it does not provide a means to adjust this artificial limit, further motivating the need for a more flexible, open-source solution.

### 2.2 Excel Inquire Add-in

The *Excel Inquire* Add-in, available only in *Office Professional Plus* and *Microsoft 365 Apps for Enterprise*, can remove some redundant content [26]. However, it lacks comprehensive batch processing capabilities because it requires manually opening each *Excel* file before applying the add-in. This approach is impractical for large datasets.

*Excel Inquire* ignores whitespace cells but can delete empty cell, column, and row definitions. Yet, in certain cases, a significant number of unnecessary elements remain. For example, consider a worksheet serving as a cover sheet, where only the first 77 rows are used and the remaining rows (78 to 65533) are hidden. Despite being hidden, all these rows and cells are defined in the XML structure, as shown in Listing 1.

Listing 1. Row 65533 in sheetX.xml

```
<row r="65533" spans="1:7" ht="0" hidden="1" customHeight="1">
  <c r="A65533" s="68" />
  <c r="B65533" s="68" />
  <c r="C65533" s="68" />
  <c r="D65533" s="68" />
  <c r="E65533" s="68" />
  <c r="F65533" s="68" />
```

```
  <c r="G65533" s="68" />
</row>
</sheetData>
```

When applying the *Excel Inquire* Add-in to such worksheets, these hidden rows and cells remain. A dialog box reports that because the default row height is `0`, the tool cannot clean the sheet, causing the operation to be skipped entirely.

### 2.3 Excel Check Performance

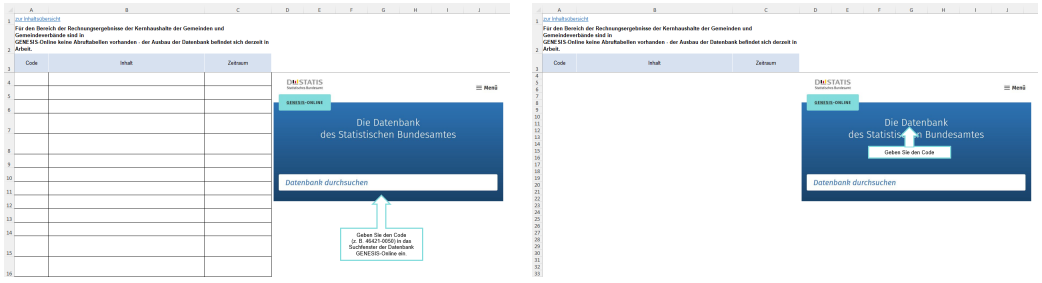
The *Check Performance* feature, introduced in *Excel* for the Web and recent desktop versions, can achieve substantial file size reductions [37, 38]. However, it often removes cells and rows that influence the spreadsheet's layout and formatting, thereby altering its visual appearance. This is unacceptable in contexts where maintaining the original look and structure of the spreadsheet is essential.

Figure 1 illustrates how *Check Performance* modifies a cover sheet by removing empty rows. While such modifications might be acceptable in some instances, they become problematic when these empty rows serve a functional purpose, such as aligning the layout for printing on A4 pages. In the example shown, the cells responsible for providing whitespace on the cover sheet (Figure 1a) are removed, as shown in Figure 1b [28].


(a) Before Applying Check Performance (b) After Applying Check Performance

Fig. 1. Comparison of a Cover Sheet Before and After Applying Check Performance

As another example, consider a form-like structure composed of several empty cells arranged to form a table, anticipating future data entry [31]. *Check Performance* removes all borders outside the active data region, resulting in the scenario shown in Figure 2, where the intended table layout is lost.




(a) Before Applying Check Performance (b) After Applying Check Performance

Fig. 2. Removal of Borders Defining an Empty Table Structure

## 3 EXCEL SHRINK

The objective of *Excel Shrink* is to reduce the size of *Excel* files by removing content that is invisible to the user and likely unintentionally included in the file. This includes empty cells, cells containing only whitespace, cells with non-visible formatting, and unused rows and columns.

### 3.1 Deletion of White Spaces

Excel users lack a straightforward method to identify which cells in a worksheet contain only whitespace characters. To highlight these whitespaces, we have marked cells containing exclusively whitespaces in red within this worksheet, which includes 4,121 such instances (see Figure 3) [30].

| | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 843 | 17 | = | Private Konsumausgaben | 1773.842 | | | | | | | | | | | | |
| 844 | | | | | | | | | | | | | | | | |
| 845 | | | Jahr: 2022 | | | | | | | | | | | | | |
| 846 | | | | | | | | | | | | | | | | |
| 847 | 1 | | Land- und Forstwirtschaft, Fischerei | 8.644 | 5.249 | 0.646 | - | 0.928 | 0.096 | - | - | - | 1.725 | - | - | - |
| 848 | 2 | | Bergbau, Verarbeitendes Gewerbe und | | | | | | | | | | | | | |
| 849 | | | Baugewerbe | 38.616 | 17.324 | 0.265 | 1.489 | 7.994 | 4.182 | 0.671 | 1.974 | 0.079 | 2.106 | - | 1.352 | 1.180 |
| 850 | 3 | | Energieversorgung | 72.984 | - | - | - | 72.984 | - | - | - | - | - | - | - | - |
| 851 | 4 | | Krafftfahrzeughandel, Instandhaltung und | | | | | | | | | | | | | |
| 852 | | | Reparatur von Kraftfahrzeugen | 117.681 | 0.055 | 0.030 | 0.044 | 0.021 | 0.029 | 0.004 | 114.264 | - | 3.128 | - | 0.102 | 0.004 |

Fig. 3. Cells containing only whitespaces are highlighted in red.

Listing 2. Cell D844 in sheet28.xml

```
<row r="844" spans="1:16" ht="14.1" customHeight="1">
  <c r="D844" s="428" t="s">
      <v>102</v>
  </c>
```

To detect these whitespaces, it is insufficient to examine the cell itself, as even a single space is stored by *Excel* in the file for recurring texts, even if the text appears only once in the entire *Excel* file. Cell D844 appears in the XML as shown in Listing 2.

The row is marked by the <row> tag with the attribute r="844", indicating that it is the 844th row. Within the row, the <c> tag represents a cell, identified by the attribute r="D844". The attribute t="s" specifies the cell type; in this case, s stands for *String* [24].

The cell's value is stored within the <v> tag, where the value 102 is specified. This value is interpreted as an index in a string table, referencing a specific text in a separate list defined in the *Excel* file, namely the sharedStrings.xml file.

Since *Excel* stores even single spaces in the Shared String Table, merely inspecting the cell's content is insufficient to detect whitespaces. By examining the shared strings, we can identify entries that contain only whitespace characters or are empty, allowing us to determine which cells can be safely deleted. To locate the value, we must search for the `<sst>` node in the `sharedStrings.xml` file, as shown in Listing 3.

Listing 3. Shared String Table (`sst`) in `sharedStrings.xml`

```
<sst count="17962" uniqueCount="732">
  <si>
    <t>062</t>
  </si>
```

The provided XML excerpt displays a portion of the Shared String Table (SST) in an *Excel* file [17]. The Shared String Table is a structure that allows for the storage and management of shared strings within an *Excel* workbook. Instead of storing each string separately in every cell, each unique string is stored once in the Shared String Table. This approach saves storage space, as cells then reference these strings via indices. Each entry in this table has an ascending index number, starting at `0` for the first string, 1 for the second, and so forth. The `count` attribute indicates the total number of strings in the table, while `uniqueCount` represents the number of unique strings. In this case, there are `17,962` strings in total, of which `732` are unique. This means that many strings in the table are used multiple times.

Within the SST, there are elements of type `<si>` (Shared Item), each containing a string. In this example, the `<t>` element displays the actual text of the string stored in the table. Here, the string `062` is represented, which can be referenced by an index in a cell.

To find the string corresponding to index `102`, we must look for the 103rd entry (since indexing starts at 0), which is shown in Listing 4.

Listing 4. Empty string at index 102 in `sst`

```
<si>
  <t xml:space="preserve"></t>
</si>
```

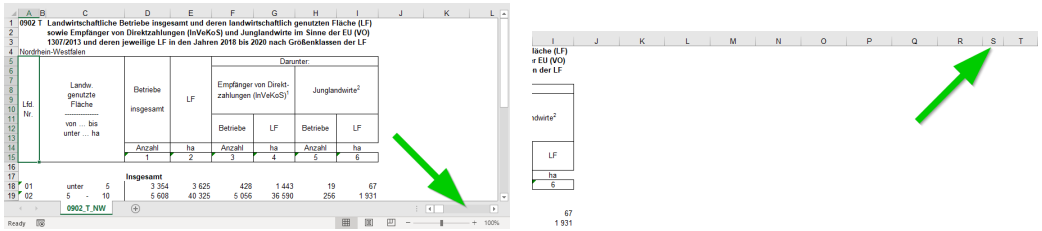
Within the `<si>` element is a `<t>` element containing the actual text of the string. In this specific case, the content of the `<t>` element is empty, indicating that no text is stored. The attribute `xml:space="preserve"` suggests that this represents a whitespace character. Therefore, all cells referencing index `102` for the reusable string can be deleted, as they contain no visible content.

### 3.2 Deletion of Empty Columns

An initial sign of excessive column definitions becomes apparent when observing the horizontal scrollbar. As illustrated in Figure 4a [27, Sheet: `0902_T_NW`], if the complete data range is visible yet the scrollbar thumb remains extremely narrow relative to the scrollable area, it suggests that numerous redundant column definitions extend well beyond the actual data range. By scrolling into these regions, users may notice that among columns with uniform widths, some columns occasionally have differing widths, as shown in Figure 4b. Such anomalies further indicate the presence of unnecessary columns contributing to file bloat.

These column definitions are specified in *Excel* within the file `sheetX.xml` under the `cols` node. In our example, the definition of the initial columns is shown in Listing 5.

In this XML excerpt, the `<cols>` element represents the definition of multiple columns in an *Excel* worksheet. Each `<col>` declaration contains specific information about one or more columns. The



(a) Very wide scrollable area and very narrow scrollbar thumb

(b) Custom column width for a column outside the data range

Fig. 4. Indicators of unnecessary column definitions extending beyond the actual data range.

`min` and `max` attributes specify the range of columns that the `<col>` element covers. For example, `min="1" max="1"` pertains only to column 1, while `min="5" max="9"` describes columns 5 through 9. The `width` attribute specifies the width of the columns in *Excel* units. The `customWidth="1"` attribute indicates that the column width is user-defined, meaning it has been manually adjusted rather than using the default value.

The first five `<col>` definitions relate to the data range. However, the last definition, `min="10" max="18"`, pertains to a redundant range. This section defines columns that lie outside the actual data range in the worksheet, specifically columns starting from column `J`, thereby representing unnecessary or unused definitions.

Listing 5. Initial column definitions

```
<cols>
  <col min="1" max="1" width="5.21875" style="39"
      customWidth="1" />
  <col min="2" max="2" width="1.77734375" style="39"
      customWidth="1" />
  <col min="3" max="3" width="20" style="39" customWidth="1" />
  <col min="4" max="4" width="11.5546875" style="39"
      customWidth="1" />
  <col min="5" max="9" width="10.5546875" style="39"
      customWidth="1" />
  <col min="10" max="18" width="11.5546875" style="54"
      customWidth="1" />
```

During our investigations, we could not determine how these column definitions originated. We hypothesize that the worksheet was duplicated, and the original worksheet contained this number of columns with the corresponding widths. However, this hypothesis is contradicted by the fact that the number of columns in the files we examined approaches *Excel*'s maximum limit of 16,384 columns. Figure 5 illustrates such a case [27, Sheet: `0902_T_NW`]. This worksheet contains 2,403 column definitions, and after excluding the five actually used column definitions, 2,398 redundant definitions remain.

Columns `WWA` and the partially visible `WWB` correspond to the 16,147th and 16,148th columns, respectively. It seems improbable that an original worksheet would require the utilization of such an extensive number of columns. Further research is required to determine how such patterns emerge in *Excel* files.


Fig. 5. Excessive column definitions in a worksheet

The last visible columns in Figure 5 are shown in Listing 6. The first column `min="16146" max="16146"` has a fixed width of `2.21875` and refers to column `WVZ`, which is visible as the column immediately to the left of `WWA` but is too narrow to display more than the letter `W`. The second column `min="16147" max="16147"` defines column `WWA` with a width of `5.21875`. The final definition `min="16148" max="16384"` covers a larger range from column `16148` to `16384`, corresponding to columns `WWB` through `XFD` (the last column in *Excel*). This explains why scrolling all the way to the right via the scrollbar lands on column `WWB`. The penultimate definition specifies where the data range ends, thereby also defining where the scrollable range ends, and the final definition simply assigns a default width to all remaining columns. Surprisingly, these column definitions extend all the way to *Excel*'s maximum limit of 16,384 columns. We have no explanation for why these user-defined column widths appear up to just before the maximum allowable column, but not exactly at it.

To remove redundant column definitions, we can implement a procedure as outlined in Algorithm 1. This algorithm identifies the actual right boundary of data and cell formats, locates redundant column definitions, and adjusts the last column definition to account for the removed columns.

**Algorithm 1** Removing Redundant Column Definitions

1: **Input:** Worksheet XML structure with column definitions
2: **Output:** Modified worksheet XML without redundant column definitions
3: Identify the actual right boundary of the data range and cell formats.
4: Find the first redundant column definition, record its `min` value as *savedMin*.
5: Remove all redundant column definitions except the last one.
6: Overwrite the `min` value of the last remaining redundant column definition with *savedMin*.
7: **return** Modified XML

Listing 6. Final column definitions in the worksheet

```
  <col min="16146" max="16146" width="2.21875" style="39"
      customWidth="1" />
  <col min="16147" max="16147" width="5.21875" style="39"
      customWidth="1" />
  <col min="16148" max="16384" width="8.77734375" style="39"
      customWidth="1" />
</cols>
```

Following these steps is crucial. We experimented with merely deleting the excess columns without ensuring that the last column definition extended to the final possible column (16,384), but this caused *Excel* to mark these worksheets as corrupted and perform certain repairs. Part of these changes included altering the default row height, resulting in some worksheets appearing visually altered, as shown in Figures 6a and 6b. On the left is the original, and on the right is the repaired version by *Excel*. For instance, row 6 has changed in height due to the repair, causing visible shifts

in the worksheet. Correctly deleting column definitions while ensuring the last column definition extends to the column limit of 16,384 prevents this error.


(a) Original worksheet before repair (b) Worksheet after repair by *Excel*

Fig. 6. Visual differences due to changes in default row height

*3.2.1 Hidden Columns.* Removing columns is not recommended if there are hidden columns. In spreadsheets like *Excel*, cells define only where text begins. If the text is longer than the cell, it typically extends over adjacent cells unless those cells also contain text. Some worksheets rely on the assumption that text will overflow into adjacent cells, causing the range of actually used cells to span multiple columns. In such cases, it's important not to delete columns outside the range of cells with values. Doing so could shift hidden columns to the next position, causing text from previous cells to overflow into them. This is depicted in Figures 7a and 7b. Since the texts are only defined in columns A and B, all cells from column C onwards are outside the range of cells with values and are thus considered for deletion. However, column C is already hidden, causing the texts to be cut off.

When *Excel Shrink* detects hidden columns, it avoids removing them to ensure that the worksheet's visual representation is not altered.


(a) Worksheet before column deletion (b) Worksheet after column deletion

Fig. 7. Effect of deleting columns when hidden columns are present

## 3.3 Deletion of Empty Cells

Some of the worksheets we examined contained vast numbers of cells without any content. The most extreme case was a worksheet with 8,141 cells containing values and 5,237,075 cells without values, meaning that only 0.15% of the stored cells actually had a value [31, Sheet: csv-71717-25]. Listing 7 shows the last row of this worksheet.

Listing 7. The last row in the worksheet csv-71717-25

```
  <row r="1048576" spans="3:7" ht="12.75" customHeight="1">
    <c r="C1048576" s="538"/>
    <c r="D1048576" s="537"/>
    <c r="E1048576" s="537"/>
    <c r="F1048576" s="537"/>
    <c r="G1048576" s="541"/>
  </row>
</sheetData>
```

Unlike the whitespaces, these cells now contain no value and therefore do not need to be cross-referenced with the sharedStrings file. However, it is possible that such empty cells may have a background color assigned, as is the case with some cover sheets used for page design or for highlighting table headers. Additionally, these cells might have borders applied.

To determine this, one must examine the `s` attribute, which stands for the style index applied to the cell. The values `s="538"`, `s="537"`, and `s="541"` correspond to indices in a lookup table where all cell styles are stored. The styles matching these style indices can be found in the `xl/styles.xml` file. The relevant excerpts are shown in Listing 8.

Listing 8. Cell styles in styles.xml for csv-71717-25

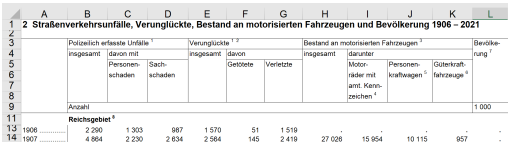
```
<cellXfs count="778">
  <!--538-->
  <xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/>
  <!--539-->
  <xf numFmtId="0" fontId="4" fillId="0" borderId="0" xfId="3"
     applyNumberFormat="1" applyFont="1"/>
  <!--542-->
  <xf numFmtId="0" fontId="1" fillId="0" borderId="0" xfId="1"
     applyNumberFormat="1" applyFont="1" applyAlignment="1">
    <alignment horizontal="right"/>
  </xf>
```

The XML structure within `<cellXfs>` contains formatting properties defined for cells in an *Excel* file. Each `<xf>` element represents a formatting rule. Similar to the Shared Strings file, formatting rules are indexed based on their order in the list, starting with the first at 0, the second at 1, and so on. For the IDs `s="538"`, `s="537"`, and `s="541"`, one must look for the 539th, 538th, and 542nd elements in the list, respectively. The attributes control the cell's appearance. The `numFmtId` attribute specifies the number format used for the cell, while `fontId` describes the font ID. `fillId` refers to the fill color, and `borderId` defines the border properties. The `xfId` references a format template from which the cell inherits properties. The attributes `applyNumberFormat`, `applyFont`, and `applyAlignment` indicate whether the corresponding number format, font, and text alignment are applied to the cell. If the `alignment` element is present, it specifies the text alignment within the cell, such as right alignment. This structure defines the formatting rules used for the display and layout of the cell contents.

Inspection of these styles reveals that these cells only apply styles that are not visible for empty cells. They use different fonts, and cell `G1048576` is additionally right-aligned. Therefore, all these cells are redundant and can be deleted accordingly.

Another case arises when the formatting rules include borders. If such cells are simply deleted, as occurred in previous versions of our script, issues arise, as seen in Figure 8 [28, Sheet: `2_(1)`]. In

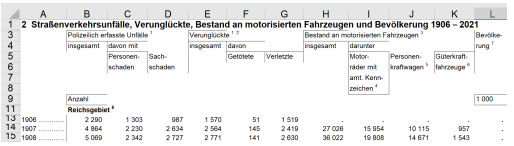
Figure 8a, all borders around the cells in the table header are visible. In Figure 8b, only the borders adjacent to cells with content remain visible. For example, the border between cells A2 and A3 is missing.



2 Straßenverkehrsunfälle, Verunglückte, Bestand an motorisierten Fahrzeugen und Bevölkerung 1906 – 2021

| | Polizeilich erfasste Unfälle | | | Verunglückte | | | Bestand an motorisierten Fahrzeugen | | | | Bevölkerung |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | insgesamt | davon mit | | insgesamt | davon | | insgesamt | darunter | | | |
| | | Personenschaden | Sachschaden | | Getötete | Verletzte | | Motorräder mit amt. Kennzeichen | Personenkraftwagen | Güterkraftfahrzeuge | |
| | Anzahl | | | | | | | | | | 1 000 |
| Reichsgebiet | | | | | | | | | | | |
| 1906 | 2 290 | 1 303 | 987 | 1 570 | 51 | 1 519 | . | . | . | . | . |
| 1907 | 4 864 | 2 230 | 2 634 | 2 564 | 145 | 2 419 | 27 026 | 15 954 | 10 115 | 957 | . |

(a) Original version with correct cell borders and spacing



2 Straßenverkehrsunfälle, Verunglückte, Bestand an motorisierten Fahrzeugen und Bevölkerung 1906 – 2021

| | Polizeilich erfasste Unfälle | | | Verunglückte | | | Bestand an motorisierten Fahrzeugen | | | | Bevölkerung |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | insgesamt | davon mit | | insgesamt | davon | | insgesamt | darunter | | | |
| | | Personenschaden | Sachschaden | | Getötete | Verletzte | | Motorräder mit amt. Kennzeichen | Personenkraftwagen | Güterkraftfahrzeuge | |
| | Anzahl | | | | | | | | | | 1 000 |
| Reichsgebiet | | | | | | | | | | | |
| 1906 | 2 290 | 1 303 | 987 | 1 570 | 51 | 1 519 | . | . | . | . | . |
| 1907 | 4 864 | 2 230 | 2 634 | 2 564 | 145 | 2 419 | 27 026 | 15 954 | 10 115 | 957 | . |
| 1908 | 5 069 | 2 342 | 2 727 | 2 771 | 141 | 2 630 | 36 022 | 19 808 | 14 671 | 1 543 | . |

(b) Corrupted version with removed borders and spacing

Fig. 8. Erroneous removal of cells with borders and deletion of rows with custom heights

Cell A2 appears in the XML as shown in Listing 9.

Listing 9. Cell A2 in sheet7.xml

```
<c r="A2" s="577"/>
```

Here, the style ID is s="577". It defines the underlying cell formatting, which in turn includes the ID for the border definition.

In the `styles.xml` file under the <cellXfs> node, we search for the <xf> node with ID 577, which corresponds to the 578th <xf> node. This is shown in Listing 10.

Listing 10. Style with implicit ID 577 in styles.xml

```
<xf numFmtId="0" fontId="8" fillId="2" borderId="5" xfId="0"
    applyFont="1" applyFill="1" applyBorder="1"
    applyAlignment="1"/>
```

This <xf> node has the `borderId` attribute set to index 5. Similar to cell styles, this index indicates that the 6th <border> node within the same `styles.xml` file is referenced.

Listing 11 shows the 6th <border> node with implicit ID 5.

Listing 11. Border node with implicit ID 5 in styles.xml

```
<borders count="12">
  <!-- ... -->
  <border>
    <left/>
    <right/>
    <top/>
    <bottom style="thin">
      <color indexed="64"/>
    </bottom>
    <diagonal/>
  </border>
```

Each <border> node contains the child nodes `left`, `right`, `top`, `bottom`, and `diagonal`. If these nodes lack attributes or child nodes, it means that the cell has no border. In this case, the `bottom` node has the attribute `style` set to "thin" and includes the `color` node with the attribute `indexed="64"`, indicating that the cell has a thin black line at the bottom. Cells identified as having borders must not

be deleted. Only cells that have neither content nor borders (i.e., their border nodes lack attributes indicating border presence) may be safely deleted.

### 3.4 Inverted Fill

During the development of *Excel Shrink*, we made an interesting discovery. We found cells that had no content and no border but a fill definition that specified no fill should be applied. As a result, these cell definitions were deleted, and the cells turned red. We discovered that the entire row containing the cell had a red fill applied, and each cell within that row had a definition specifying that no fill should be applied to override the red fill color. As a user, one does not have any means to identify this by looking at the sheet. All the visible cells had the overridden fill color, and the rest of the cells were hidden behind hidden columns. Only after unhiding those columns did the issue become evident. Situations like these make preserving the visual appearance especially difficult.

We cannot simply decide to delete all cells that have a style applied that tells the cell to apply no fill because such a scenario might be useful in some situations. For example, if the whole row or column is intended to have a certain style and only some cells should apply no style as an exception to the rule. At the same time, in some scenarios, this might not be intended. The file becomes exceptionally large because some rows inadvertently have a style applied, and all cells in those rows up to the limit of possible columns define that this style should be overridden by no style.

To reduce the file size while preventing such changes to the display, we proceed as follows. First, we determine the style of the column and the row in which the cell is located. If either the column or row has a background color defined, we check whether the cell defines the same background color. In this case, we delete the cell. If the background color information of the cell differs from that of the column or row, we retain the cell. The same applies analogously to borders. If the column or row defines a border, we check whether the cell defines the same border. In this case, we delete the cell. If the border information of the cell differs from that of the column or row, we retain the cell.

## 4 DELETION OF EMPTY ROWS

Similar to cells without content but with visible formatting, rows may contain no defined cells but still have a defined height. Such rows must not be deleted because their height can affect the worksheet's layout. If such a row is deleted, an issue occurs as depicted in Figure 8b compared to the original in Figure 8a. The spacing between the header and the table shrinks because the intermediate row, which served as spacing between the header and the table, is deleted, and the default row height for the worksheet is applied, which is smaller than the custom height of the row. This row is shown in Listing 12.

Listing 12. Row with custom height

```
<row r="2" spans="1:12" ht="8.25" customHeight="1">
```

The attribute `customHeight` means that this row has been assigned a user-defined height, specified by the `ht` attribute: `8.25`. Therefore, this row must not be deleted. However, during our examination of the worksheets, we found many that contained an immense number of empty rows, which had defined heights.

In the most extreme case in our analysis, a worksheet contained 1,048,576 defined rows. Of these, only 94 rows had content, which corresponds to approximately 0.00896% of the total rows [29, Sheet: `Tabelle 1.2.1`]. The remaining 1,048,482 rows had a defined height but were outside the data range.

Listing 13. The last row in the worksheet

```
<row r="1048576" ht="12.75" hidden="1" customHeight="1"/>
```

After the 96th row, which is the last one with content, there are 1,048,482 rows that are identical to the last row shown in Listing 13, except for the `r` attribute. These rows all have the same height and are hidden, as indicated by the `hidden="1"` attribute. Although they are not displayed to the user, they have a user-defined height of 12.75 units, which is why they are still stored in the file. Row 1,048,576 corresponds to the last row, which is the maximum number of rows allowed in *Excel*. If *Excel* allowed a higher number of rows, the wasted file storage would be significantly greater.

We cannot rely on a row simply not having a defined height to decide whether it can be deleted. However, we also cannot simply delete rows that have a user-defined height, as they may be important for the worksheet's layout. Therefore, we must find another method to identify empty rows that can be safely deleted.

To ensure that deleting empty rows does not alter the worksheet's visual representation, we adopt a strategy similar to that used for removing columns. First, we determine the actual range where values, background colors, or borders are defined. Then, we iterate through all empty rows and delete them only if they lie outside this range. This approach allows us to significantly reduce the file size of some worksheets without altering their content or appearance.

### 4.1 Deletion of Redundant Print Areas and Print Titles

Excel allows users to define the area of a worksheet that should be printed on paper. This area can be manually defined or automatically set by *Excel*. In both cases, this area is stored in the `workbook.xml` file under the `<definedNames>` node [16]. Typically, this area is labeled as `_xlnm.Print_Area` and `_xlnm.Print_Titles`. However, these areas and titles are often also stored under the names `Print_Area` and `Print_Titles`. When *Excel* opens the file, it renames the entries `Print_Area` and `Print_Titles` to `_xlnm.Print_Area` and `_xlnm.Print_Titles`, respectively. This process creates a problem when there is already a *Defined Name* for the same worksheet with both `Print_Area` and `_xlnm.Print_Area`, as seen in Listing 14[29, `xl/workbook.xml`].

Listing 14. Redundant Print Area entries

```
<definedName name="_xlnm.Print_Area" localSheetId="12">'Tabelle
    2.1.1'!$A$1:$N$64748</definedName>
<!-- ... -->
<definedName name="Print_Area" localSheetId="12">'Tabelle
    2.1.1'!$A$1:$N$93</definedName>
```

A similar issue occurs with `Print_Titles` and `_xlnm.Print_Titles`. In these cases, *Excel* cannot simply rename the entries without causing duplicates. Instead, it displays a dialog box prompting the user to select a new name for each redundant *Defined Name* upon file opening.

If the user renames the first entry and clicks *OK*, the dialog reappears for each subsequent redundant entry. This process repeats until all duplicates are resolved. The user cannot reuse the same name for different *Defined Names* on the same worksheet, which further complicates the process. Moreover, this dialog prevents automated file opening via VBA to leverage tools like the *Inquire* Add-in or the *Excel Performance* tab for batch file size reduction, since the dialog interrupts any automation.

In the examined files, we observed up to 60 redundant `Print_Area` and 55 redundant `Print_Titles` entries. Renaming these entries confers no actual benefit, as the relevant print areas and titles are already correctly migrated. Users may believe they are repairing *Print Areas* or `Print Titles`, but in reality, they are only renaming redundant *Defined Names*. To mitigate this issue and spare users from manually renaming entries, *Excel Shrink* automatically removes these redundant definitions.

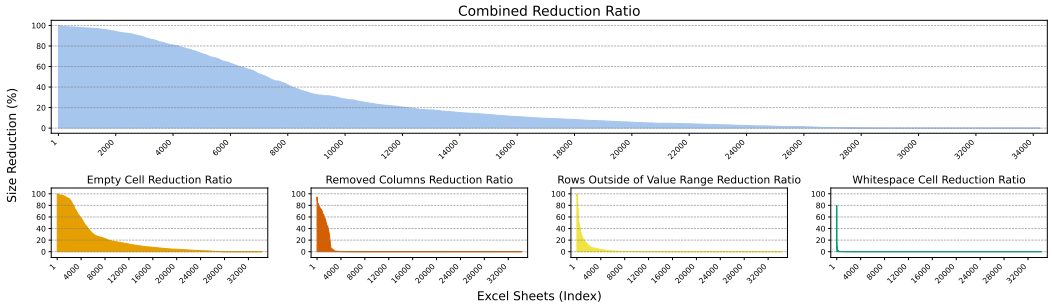



Fig. 9. Distribution of size reduction percentages for Excel sheets

### 4.2 Reporting Reduction Results

To quantify the impact of each reduction step—such as removing whitespace, empty cells, columns, and rows—we compare file sizes before and after each optimization. Specifically, we convert the worksheet's XML structure into a string representation and record its length. After performing a reduction step, we measure the string length again and calculate the difference. The resulting percentage indicates how much size reduction that particular step achieved. These results are recorded in a CSV file for further analysis [9].

## 5 EVALUATION

In this evaluation, we processed all *Excel* files obtained from the German Federal Statistical Office (*Destatis*) [2, 4]. Detailed information on how these files were acquired via a custom web scraping approach can be found in Appendix A.

Our primary objective was to measure the degree of data reduction achieved by *Excel Shrink*, focusing on metrics such as overall size reduction, removal of empty cells, elimination of whitespace cells, deletion of rows outside the value range, and removal of unnecessary columns. Figure 9 presents the distribution of reduction rates (in percentages) for individual worksheets across all these categories. Some worksheets approached a 100% reduction in file size. In particular, one worksheet exceeded 99.99%, 16 worksheets surpassed 99.9%, and 210 worksheets achieved reductions above 99%. On average, worksheets were reduced by 25%. Most of these improvements resulted from removing rows that lay outside the actual data range.

To further assess performance improvements, we identified the largest available *Destatis* file [31]. Before processing, this file measured 85.2 MB in its compressed *Excel* (.xlsx) format and expanded to 911 MB uncompressed. After applying *Excel Shrink*—a process taking a median of 484 seconds over 10 runs—the compressed file size dropped to 2.76 MB, with an uncompressed size of just 20.6 MB [33].

Subsequent tests were performed using both the Dart `excel` package [3, 19] and the Python `OpenPyXL` package [12, 21]. Prior to shrinking, the Dart `excel` package required 794.7 seconds on its first run to open and read values, then failed with an out-of-memory exception on the second run [5]. The `OpenPyXL` package required a median of 301 seconds over 10 runs [6].

In contrast, after applying *Excel Shrink*, the Dart `excel` package successfully ran 10 consecutive tests without error, with a median time of 29.8 seconds [8], and `OpenPyXL` required a median of 5.2 seconds [7], demonstrating a substantial performance improvement. Figure 10 shows these results, highlighting the effectiveness of *Excel Shrink* in reducing processing times.

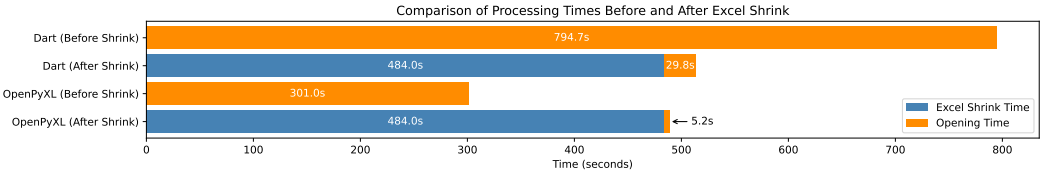



Fig. 10. Comparison of processing times before and after applying *Excel Shrink*.

Our evaluation demonstrates that *Excel Shrink* effectively eliminates redundant, empty, or irrelevant data across a diverse set of *Excel* worksheets. Significant reductions—exceeding 90% in several cases—were achieved in sheets containing a high proportion of empty cells, or irrelevant rows and columns. These results highlight the robustness of our reduction methods in processing diverse data formats and structures. Moreover, the performance improvements in file opening times underscore the practical benefits of using *Excel Shrink* to enhance data processing efficiency.

## 6 EXPERIMENTAL SETUP

To evaluate the behavior of *Excel* regarding how the files are interpreted and saved, as well as to test the behavior of the *Add-in* and the new *Check Performance* feature, we tested with two distinct versions of *Excel* from *Microsoft Office*: *Microsoft Office LTSC Professional Plus 2021* and *Microsoft Excel for Microsoft 365 MSO (Version 2409 Build 16.0.18025.20030) 64-bit*. These versions were selected to evaluate compatibility and performance across different Excel environments.Our test machine was an *Intel Core i5-10210U* quad-core CPU desktop machine running at 1.60 GHz with 16 GB of RAM.

## 7 OUTLOOK

While *Excel Shrink* can significantly reduce file sizes and thereby expedite the opening of *Excel* files through libraries like `OpenPyXL` or the Dart `excel` package, there is still room for further optimization. The current implementation relies on an XML DOM parser, which can be replaced with a StAX or SAX parser to lower memory usage and improve processing speed. Such a streaming-based parser would consume far less memory, allowing multiple processes or threads to operate on different worksheets in parallel, thus further reducing the overall processing time.

## 8 CONCLUSION

We have presented *Excel Shrink*, an open-source tool that significantly reduces the file sizes of *Excel* sheets from *Destatis* by eliminating unnecessary content. Our analysis identified common practices that lead to bloated *Excel* files and demonstrated how targeted removal of redundant data can achieve reductions of up to 99.99% without altering the spreadsheets' appearance.

Our tool outperforms existing solutions in both file size reduction and preservation of formatting. By making *Excel Shrink* available to the community, we hope to facilitate more efficient data processing and analysis of large-scale *Excel* datasets.

## REFERENCES

[1] Rui Abreu, Jácome Cunha, Joao Paulo Fernandes, Pedro Martins, Alexandre Perez, and Joao Saraiva. 2014. Faultysheet detective: When smells meet fault localization. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 625–628.

[2] Anonymous. [n.d.]. Destatis XLSX Files Shrunk with Excel Shrink. https://anonymous.4open.science/r/shrinked_destatis_xlsx_files-4E27/. Accessed: 2024-12-07.

[3] Anonymous. [n.d.]. Open Large XLSX File with Dart - Test Script. https://anonymous.4open.science/r/results-of-excel-shrink-on-destatis-xlsx-files-7CC5/open_large_xlsx_file_with_dart/test/open_large_xlsx_file_with_dart_test.dart. Accessed: 2024-12-08.

[4] Anonymous. [n.d.]. Original Destatis XLSX Files Before Shrinking with Excel Shrink. https://anonymous.4open.science/r/original_destatis_xlsx_files-5B7B/. Accessed: 2024-12-07.

[5] Anonymous. [n.d.]. Results for Decoding Large Excel Workbook with the Dart 'excel' Package. https://anonymous.4open.science/r/results-of-excel-shrink-on-destatis-xlsx-files-7CC5/testdata/run_times_to_open_worksheet_excel_package.csv. Accessed: 2024-12-08.

[6] Anonymous. [n.d.]. Results for Decoding Large Excel Workbook with the OpenPyXL. https://anonymous.4open.science/r/results-of-excel-shrink-on-destatis-xlsx-files-7CC5/testdata/run_times_to_open_worksheet_openpyxl_package.csv. Accessed: 2024-12-08.

[7] Anonymous. [n.d.]. Results for Decoding Shrunk Excel Workbook with OpenPyXL. https://anonymous.4open.science/r/results-of-excel-shrink-on-destatis-xlsx-files-7CC5/testdata/run_times_to_open_shrinked_worksheet_openpyxl_package.csv. Accessed: 2024-12-08.

[8] Anonymous. [n.d.]. Results for Decoding Shrunk Excel Workbook with the Dart 'excel' Package. https://anonymous.4open.science/r/results-of-excel-shrink-on-destatis-xlsx-files-7CC5/testdata/run_times_to_open_shrinked_worksheet_excel_package.csv. Accessed: 2024-12-08.

[9] Anonymous. [n.d.]. Results of Excel Shrink on Destatis XLSX Files. https://anonymous.4open.science/r/excel-shrink-results-7468/shrink_results.csv. Accessed: 2024-12-08.

[10] Anonymous. 2024. Excel Shrink - An open-source tool designed to optimize Excel files by eliminating redundant content without altering their visual appearance or functionality. https://anonymous.4open.science/r/excel-shrink-F754/README.md.

[11] Anonymous. 2024. Excel Shrink UI - An open-source graphical user interface for Excel Shrink. https://anonymous.4open.science/r/excel-shrink-ui-BE5C/README.md.

[12] Anonymous. 2024. Open Large XLSX File with Python - Test Script. https://anonymous.4open.science/r/results-of-excel-shrink-on-destatis-xlsx-files-7CC5/open_large_xlsx_file_with_python/open_large_xlsx_file_with_python_test.py. Accessed: 2024-12-10.

[13] Mangesh Bendre, Bofan Sun, Ding Zhang, Xinyan Zhou, Kevin ChenChuan Chang, and Aditya Parameswaran. 2015. Dataspread: Unifying databases and spreadsheets. In *Proceedings of the VLDB Endowment International Conference on Very Large Data Bases*, Vol. 8. NIH Public Access, 2000.

[14] Mangesh Bendre, Tana Wattanawaroon, Kelly Mack, Kevin Chang, and Aditya Parameswaran. 2019. Anti-freeze for large and complex spreadsheets: Asynchronous formula computation. In *Proceedings of the 2019 International Conference on Management of Data*. 1277–1294.

[15] George Chalhoub and Advait Sarkar. 2022. "It's freedom to put things where my mind wants": Understanding and improving the user experience of structuring data in spreadsheets. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–24.

[16] Microsoft Corporation. 2024. Workbook Class (DocumentFormat.OpenXml.Spreadsheet) | Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.workbook?view=openxml-3.0.1. Accessed on October 7, 2024.

[17] Microsoft Corporation. 2024. Working with the shared string table | Microsoft Learn. https://learn.microsoft.com/en-us/office/open-xml/spreadsheet/working-with-the-shared-string-table?tabs=cs. Accessed on October 7, 2024.

[18] Jácome Cunha, Joao Paulo Fernandes, Pedro Martins, Jorge Mendes, and Joao Saraiva. 2012. Smellsheet detective: A tool for detecting bad smells in spreadsheets. In *2012 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 243–244.

[19] Kawal Desai. 2024. Excel Dart Package. https://pub.dev/packages/excel. Accessed: 2024-12-03.

[20] Peter Deutsch. 1996. *DEFLATE compressed data format specification version 1.3*. Technical Report.

[21] OpenPyXL Developers. 2024. *OpenPyXL Documentation*. OpenPyXL Project. https://openpyxl.readthedocs.io/en/stable/ Version 3.1.2 (as of 2024).

[22] Ecma International. 2021. *ECMA-376: Office Open XML File Formats* (5th ed.). Technical Report ECMA-376. Ecma International. https://ecma-international.org/publications-and-standards/standards/ecma-376/ ISO/IEC 29500, Technical Committee TC45.

[23] Kelly Mack, John Lee, Kevin Chang, Karrie Karahalios, and Aditya Parameswaran. 2018. Characterizing scalability issues in spreadsheet software using online forums. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–9.

[24] Microsoft. 2014. *Open XML SDK 2.5 for Office*. Microsoft Press.

[25] Neuxpower. 2024. NXPowerLite Desktop - File Compressor Software. https://neuxpower.com/nxpowerlite-desktop/. Accessed on October 7, 2024.

[26] Patrick O'Beirne. 2014. Excel 2013 Spreadsheet Inquire. *arXiv preprint arXiv:1401.7586* (2014).

[27] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2021. Förderprogramme - Landwirtschaftszählung 2020 (Support Programs - Agricultural Census 2020). https://www.destatis.de/DE/Themen/Branchen-Unternehmen/Landwirtschaft-Forstwirtschaft-Fischerei/Landwirtschaftliche-Betriebe/Publikationen/Downloads-Landwirtschaftliche-Betriebe/foerderprogramme-5411206209004.html Accessed: 2024-12-08.

[28] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2022. Verkehrsunfälle - Zeitreihen 2021 (Traffic Accidents - Time Series 2021) (Letzte Ausgabe - berichtsweise eingestellt). https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/verkehrsunfaelle-zeitreihen-pdf-5462403.html. Accessed on October 7, 2024.

[29] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2022. Wanderungen - Fachserie 1 Reihe 1.2 - 2021 (Migration - Series 1, Series 1.2 - 2021) (Letzte Ausgabe - berichtsweise eingestellt). https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Bevoelkerung/Wanderungen/Publikationen/Downloads-Wanderungen/wanderungen-2010120217004.html. Accessed on October 7, 2024.

[30] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2023. Private Konsumausgaben und Verfügbares Einkommen - 1. Vierteljahr 2023 (Private Consumption Expenditures and Disposable Income - 1st Quarter 2023) (Letzte Ausgabe - berichtsweise eingestellt). https://www.destatis.de/DE/Themen/Wirtschaft/Volkswirtschaftliche-Gesamtrechnungen-Inlandsprodukt/Publikationen/Downloads-Inlandsprodukt/konsumausgaben-pdf-5811109.html. Accessed on October 7, 2024.

[31] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2023. Statistischer Bericht - Rechnungsergebnisse der Kernhaushalte der Gemeinden und Gemeindeverbände 2021 (Statistical Report - Financial Results of the Core Budgets of Municipalities and Municipal Associations 2021). https://www.destatis.de/DE/Themen/Staat/Oeffentliche-Finanzen/Ausgaben-Einnahmen/Publikationen/Downloads-Ausgaben-und-Einnahmen/statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005.html. Accessed on October 7, 2024.

[32] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2024. robots.txt. https://www.destatis.de/robots.txt. Accessed on October 7, 2024.

[33] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2024. Shrunk File of: Statistischer Bericht - Rechnungsergebnisse der Kernhaushalte der Gemeinden und Gemeindeverbände 2021 (Statistical Report - Financial Results of the Core Budgets of Municipalities and Municipal Associations 2021). https://anonymous.4open.science/r/shrinked_destatis_xlsx_files-4E27/statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005.xlsx. Accessed: 2024-12-08.

[34] Raymond R Panko. 1998. What we know about spreadsheet errors. *Journal of Organizational and End User Computing (JOEUC)* 10, 2 (1998), 15–21.

[35] Sajjadur Rahman, Mangesh Bendre, Yuyang Liu, Shichu Zhu, Zhaoyuan Su, Karrie Karahalios, and Aditya G Parameswaran. 2021. NOAH: interactive spreadsheet exploration with dynamic hierarchical overviews. *Proceedings of the VLDB Endowment* 14, 6 (2021), 970–983.

[36] Sajjadur Rahman, Kelly Mack, Mangesh Bendre, Ruilin Zhang, Karrie Karahalios, and Aditya Parameswaran. 2020. Benchmarking spreadsheet systems. In *Proceedings of the 2020 acm sigmod international conference on management of data*. 1589–1599.

[37] Prash Shirolkar. 2022. Check Performance in Excel for the Web. https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/check-performance-in-excel-for-the-web/ba-p/4216095. Microsoft 365 Insider Blog, Accessed: 2024-04-27.

[38] Prash Shirolkar. 2024. *Make Your Workbooks More Performant with Check Performance in Excel for Windows.* https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/make-your-workbooks-more-performant-with-check-performance-in/ba-p/4224276 Accessed: 2024-10-18.

[39] Alaaeddin Swidan, Felienne Hermans, and Ruben Koesoemowidjojo. 2016. Improving the performance of a large scale spreadsheet: a case study. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. IEEE, 673–677.

[40] Dixin Tang, Fanchao Chen, Christopher De Leon, Tana Wattanawaroon, Jeaseok Yun, Srinivasan Seshadri, and Aditya G Parameswaran. 2023. Efficient and Compact Spreadsheet Formula Graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2634–2646.

[41] Yuzhang Tian, Jianbo Zhao, Haoyu Dong, Junyu Xiong, Shiyu Xia, Mengyu Zhou, Yun Lin, José Cambronero, Yeye He, Shi Han, et al. 2024. Spreadsheetllm: Encoding spreadsheets for large language models. *arXiv preprint arXiv:2407.09025* (2024).

## A SCRAPING DESTATIS

The *Destatis* website does not provide an API for searching publications, making it necessary to interact with the website directly. To automate this process, we developed a web scraper that searches for relevant publications while minimizing the number of requests to the server. The web scraping and downloading process is outlined in Algorithm 2. The URL structure of the *Destatis* website is as follows, with query parameters appended to the base path:

- /SiteGlobals/Forms/Suche/Expertensuche_Formular.html:
  This is the base path of the URL, which points to the expert search form on the *Destatis* website. It serves as the primary endpoint where the search query is submitted.
- templateQueryString=2000:
  This query parameter specifies the search year. In this case, the search is focused on publications from the year 2000. By adjusting this value, different years can be searched.
- gtp=474_list%253D2:
  This parameter handles pagination. It points to a specific page of search results. The value `%253D` is a doubly encoded sequence, where `%253D` is the URL-encoded form of `%3D`, which itself encodes the equal sign (=). After decoding twice, the query parameter becomes `gtp=474_list=2`. The number after the second equal sign corresponds to the current page index in the pagination of the search results.
- documentType_=publication:
  This parameter filters the results by the type of document. By setting this to `publication`, the search is limited to publications, where *Excel* files are usually attached.
- resultsPerPage=100:
  This parameter determines how many results should be displayed per page. In this case, it is set to `100`, which is the maximum number of results that can be retrieved in a single page request.

Through extensive testing, we discovered that *Excel* files are exclusively found in the "Publication" document type. By using the GET parameter `documentType_=publication`, we were able to significantly narrow down our search results.

Initially, we assumed that searching for the term `xlsx` would identify all publications containing *Excel* file attachments. However, this search only returned about 100 results, far fewer than the actual number of *Excel* files available on the *Destatis* website. Furthermore, there is no option to retrieve a complete list of all publications ever created. To overcome this, we refined our strategy by searching based on publication years. A publication should appear in the search results when queried by its publication year. Although this approach returns some irrelevant results—where the publication year might appear in the text or a file attachment—the number of requests is greatly reduced by caching and reusing links to the *Excel* file attachments. This ensures that the same files are not downloaded multiple times.

The search results on the *Destatis* website, like many others, are paginated, with the interface offering the ability to display 10, 20, or 30 entries per page. The number of results per page can be adjusted using the GET parameter `resultsPerPage=30`. While the interface does not provide an option for more than 30 results per page, we found that manually setting this parameter allowed us to request up to 100 results per page. Through testing, we confirmed that 100 is the maximum number of results that can be returned at once. We set the `resultsPerPage` parameter accordingly to reduce the number of requests further.

Once the results for a page are fetched by the web scraper, we simulate clicking the "next page" link to load subsequent results. The XPath expression `//a[@class='forward button']/@href` is used to identify the link using the CSS class `forward button`.

**Algorithm 2** Web Scraping and Downloading Excel Files with Caching

**Input:** Base URL `B`, form URL `F`, download folder `D`, CSV cache file `C`, crawl delay (30 seconds)
**Initialize:** Cached URLs set `U`, and load file records from CSV `C`
Create folder `D` if it does not exist
{— Scraping Process —}
**for** each year in years **do**
  Construct the search URL using year and base parameters
  **while** there are more pages to fetch **do**
    Send HTTP GET request to search URL
    Parse HTML to extract `.xlsx` links
    Filter links that are not already in the cache `U`
    **for** each new link **do**
      Extract file name from link
      Add new link and file name to file records
      Append new records to cache `C`
    **end for**
    Save the updated file records back to CSV `C`
    Check for the next page button
    **if** next page exists **then**
      Update URL to the next page
      Wait for `CRAWL_DELAY` seconds
    **else**
      Exit the loop
    **end if**
  **end while**
**end for**
{— Download Process —}
**for** each new file record **do**
  Download file from the URL
  Save it in the download folder `D`
  Wait for `CRAWL_DELAY` seconds
**end for**

Once we have queried each publication year from `2000` to `2024`, and the file attachments have been extracted from the result pages until no further "next page" button is available, the scraping process for the *Excel* files is complete.

### A.1 Respecting Crawl Delay

To minimize the load on the *Destatis* website, we adhered to the `crawl-delay` directive specified in the site's `robots.txt` file [32], pausing for 30 seconds between requests. This delay ensures that our scraper operates within the guidelines provided by *Destatis*.

Received 17 October 2024; revised ; accepted